SASD GmbH

Artefakt-Anlage-Leitfaden

downloads-sasd | Version 0.1.0

Ausführliche Arbeitsanleitung zum Anlegen neuer Downloads in **downloads.sasd.de**. Die Anleitung beschreibt vollständig, welche Datei wohin gehört, welche Metadaten gepflegt werden müssen, wie die Validierung abläuft und wie ein neues Artefakt sicher veröffentlicht wird.

Dokumenttyp	Betriebs- und Pflegeleitfaden für Artefaktpflege
Zielgruppe	SASD-Administration / Produktpflege / Redaktionsverantwortliche
Projektbezug	downloads-sasd Repository und geplante Plattform downloads.sasd.de
Stand	2026-03-31	Erstfassung für v0.1.0

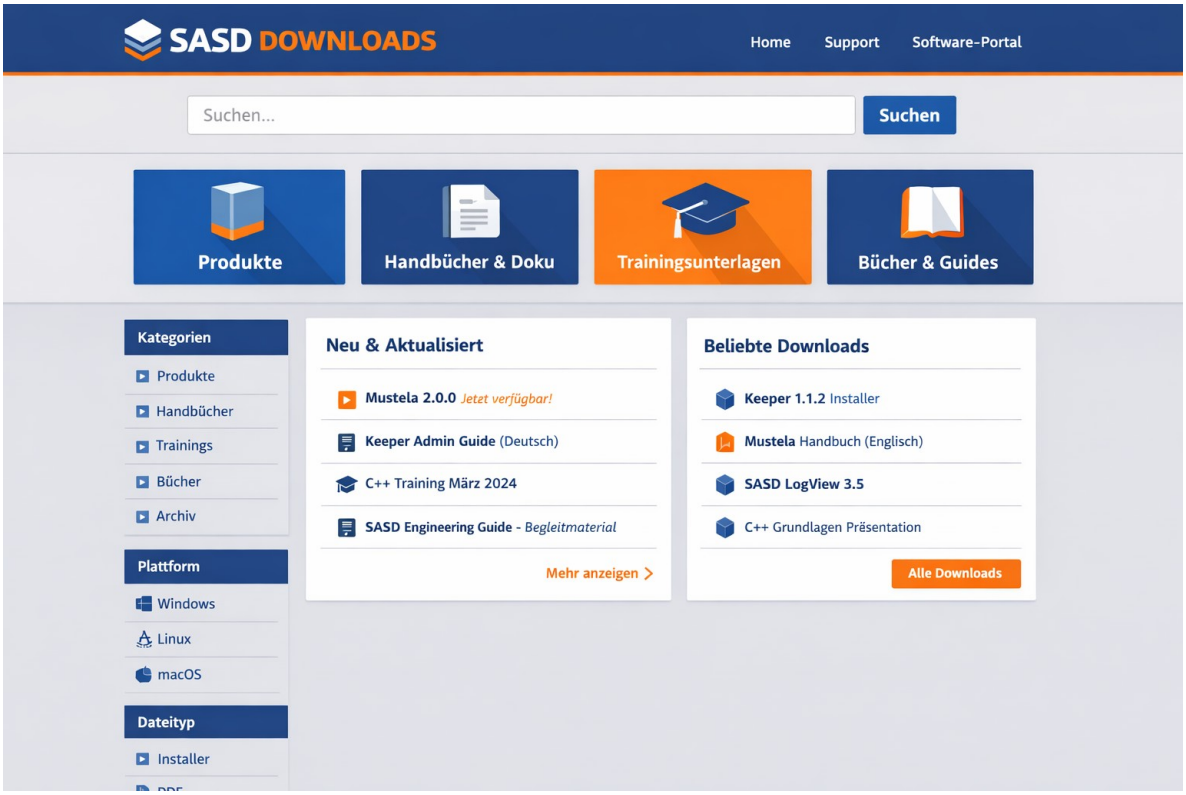

Abbildung 1: Zielbild der Startseite von downloads.sasd.de im SASD-Stil.

1. Zweck und Geltungsbereich

Dieses Dokument beschreibt den vollständigen Arbeitsablauf, um in **downloads-sasd** ein neues Artefakt so anzulegen, dass es nach der Kataloggenerierung auf **downloads.sasd.de** gefunden, angezeigt und heruntergeladen werden kann. Die Anleitung deckt sowohl den Normalfall „neues Artefakt für ein bestehendes Produkt“ als auch den erweiterten Fall „neues Produkt mit erstem Artefakt“ ab.

Wann Sie dieses Dokument benutzen sollten

Verwenden Sie diesen Leitfaden immer dann, wenn ein neues ZIP-Paket, PDF-Handbuch, Trainingsdokument, Companion-Paket oder sonstiges Download-Artefakt in den Katalog aufgenommen werden soll. Für allgemeine Serverinstallation und Deployment des Gesamtsystems ist weiterhin der separate Setup-Leitfaden zuständig.

2. Architektur in einem Satz

Ein Artefakt wird zuerst als physische Datei unter storage/files abgelegt, danach als Metadatensatz in data/catalog/source/artifacts.json beschrieben, anschließend per Validierung geprüft und schließlich über das CLI-Skript in den generierten Katalog überführt.

Baustein	Aufgabe	Für die Artefakt-Anlage relevant?
storage/files	Enthält die tatsächlich auszuliefernden Dateien wie ZIP, PDF oder Begleitmaterial.	Ja, hier wird die neue Datei physisch abgelegt.
data/catalog/source/artifacts.json	Enthält die redaktionell gepflegten Metadaten aller Artefakte.	Ja, hier wird der neue Eintrag angelegt.
data/catalog/source/products.json	Beschreibt Produkte, deren Detailseiten und Grundinformationen.	Nur nötig, wenn das Artefakt zu einem neuen Produkt gehört.
cli/validate-catalog.php	Prüft Pflichtfelder und Dateipfade.	Ja, vor jeder Veröffentlichung ausführen.
cli/rebuild-catalog.php	Erzeugt catalog.json inklusive Dateiname, Größe und SHA-256.	Ja, nach erfolgreicher Validierung ausführen.
data/catalog/generated/catalog.json	Maschinell generierter Katalog für die Anwendung.	Wird indirekt aktualisiert; nicht manuell pflegen.

3. Überblick über den Arbeitsablauf

- Datei technisch vorbereiten: endgültigen Dateinamen vergeben, Inhalt prüfen, Version und Sprache festlegen.
- Passenden Zielordner unter storage/files bestimmen und die Datei dort ablegen.
- Neuen JSON-Eintrag in data/catalog/source/artifacts.json anlegen.
- Bei einem neuen Produkt zusätzlich products.json erweitern.
- Katalog validieren und Fehler beheben.
- Generierten Katalog neu aufbauen.
- Lokal testen: Detailseite, Listenansicht, Suche und Download-Link prüfen.
- Änderung deployen und nach Deployment erneut gegen die Zielumgebung gegenprüfen.

Wichtige Grundregel

Die eigentliche Wahrheit besteht immer aus zwei Teilen: der physischen Datei und dem Metadatensatz. Ein JSON-Eintrag ohne Datei ist ungültig. Eine Datei ohne JSON-Eintrag ist unsichtbar und erscheint nicht im Katalog.

4. Vorbereitende Entscheidungen

Bevor Sie Dateien kopieren oder JSON bearbeiten, sollten Sie bewusst festlegen, zu welcher Gruppe das Artefakt gehört, welche Version angezeigt werden soll, in welcher Sprache es vorliegt und ob es dem Status current, deprecated oder archived zugeordnet werden soll. Diese Entscheidungen bestimmen später, wie das Artefakt in Listen, Filtern und Detailseiten erscheint.

5. Artefakttyp und Zielpfad bestimmen

Die Verzeichnisstruktur unter storage/files spiegelt die fachliche Einordnung. Produkte liegen unter products/<produkt-slug>/<version>/, Trainingsmaterial unter training/<slug>/<version>/ und buchbezogene Begleitdateien unter books/<slug>/<version>/. Die Pfadangabe im JSON-Eintrag muss exakt relativ zu storage/files notiert werden.

Fall	Empfohlener Speicherort	Beispiel
Softwarepaket eines Produkts	products/<productSlug>/<version>/	products/mustela/2.1.0/
Handbuch oder Guide eines Produkts	products/<productSlug>/<version>/	products/keeper/1.2.0/
Trainingsunterlage	training/<trainingSlug>/	training/cpp-course/2026-06/

Fall	Empfohlener Speicherort	Beispiel
	<version>/	
Buch-Begleitmaterial	books/<bookSlug>/<version>/	books/sasd-engineering-guide/2026-06/

6. Datei unter storage/files anlegen

Legen Sie den Zielordner an, falls er noch nicht existiert, und kopieren Sie anschließend das neue Artefakt hinein. Verwenden Sie konsistente Dateinamen mit Produktname, Version, ggf. Sprache und Dateityp. Dateinamen sollten klar, stabil und ohne Leerzeichen gestaltet sein.

```
mkdir -p storage/files/products/mustela/2.1.0
cp /quelle/mustela-manual-2.1.0-en.pdf    storage/files/products/mustela/2.1.0/mustela-manual-2.1.0-en.pdf
```

Namenskonvention

Empfohlen sind kleingeschriebene, technisch stabile Dateinamen mit Bindestrichen. Gute Beispiele sind mustela-2.1.0.zip, mustela-handbuch-2.1.0-de.pdf oder cpp-training-2026-06-workbook-de.pdf. Vermeiden Sie Leerzeichen, Umlaute im Dateinamen und wechselnde Schreibweisen desselben Produkts.

7. Metadatensatz in artifacts.json anlegen

Öffnen Sie data/catalog/source/artifacts.json und ergänzen Sie ein neues JSON-Objekt. Achten Sie auf gültige Kommasetzung und konsistente Feldnamen. Der neue Eintrag kann an fachlich passender Stelle oder am Ende der Liste eingefügt werden; technisch entscheidend ist nur gültiges JSON.

Feld	Bedeutung	Hinweise
id	Interne, stabile Kennung des Artefakts.	Soll eindeutig sein; idealerweise Produkt, Version und Typ kombinieren.
slug	URL-freundlicher Schlüssel.	Wird für Detailseiten verwendet; nur Kleinbuchstaben, Ziffern und Bindestriche.
productSlug	Zuordnung zu einem Produkt.	Bei produktbezogenen Artefakten Pflicht; bei Training/Büchern null möglich.

Feld	Bedeutung	Hinweise
group	Fachgruppe des Artefakts.	Im Projekt verwendet: products, documentation, training, books.
type	Spezifischer Artefakttyp.	Beispiele: package, installer, manual, guide, workbook, slides, companion.
title	Mehrsprachiger Titel.	Als Objekt mit mindestens de und en hinterlegen.
description	Mehrsprachige Kurzbeschreibung.	Ebenfalls als Objekt mit de und en.
version	Angezeigte Versionskennung.	Kann SemVer oder Trainingsstand wie 2026-06 sein.
language	Sprache des Inhalts.	Typisch de, en oder multi.
platform	Technische Zielplattform.	Typisch windows, linux, macos oder neutral.
status	Lebenszyklus-Status.	Typisch current, deprecated oder archived.
releaseDate	Sichtbares Veröffentlichungsdatum.	Format YYYY-MM-DD.
path	Relativer Dateipfad ab storage/files.	Muss exakt zur physischen Datei passen.
featured / popular	Steuert Hervorhebungen in der Oberfläche.	Boolean true oder false.
tags	Such- und Kontextbegriffe.	Kurze, konsistente Stichwörter verwenden.

```
{
 "id": "mustela-2.1.0-manual-en",
 "slug": "mustela-manual-2-1-0-en",
 "productSlug": "mustela",
 "group": "documentation",
 "type": "manual",
 "title": {
  "de": "Mustela Manual (Englisch)",
```

```
  "en": "Mustela Manual (English)"
 },
 "description": {
  "de": "Englisches Handbuch für Mustela 2.1.0.",
  "en": "English manual for Mustela 2.1.0."
 },
 "version": "2.1.0",
 "language": "en",
 "platform": "neutral",
 "status": "current",
 "releaseDate": "2026-04-15",
 "path": "products/mustela/2.1.0/mustela-manual-2.1.0-en.pdf",
 "featured": false,
 "popular": true,
 "tags": ["manual", "en", "docs"]
}
```

8. Falls nötig: products.json für ein neues Produkt ergänzen

Wenn das neue Artefakt zu einem bereits vorhandenen Produkt gehört, ist dieser Schritt nicht nötig. Wenn jedoch ein komplett neues Produkt erstmals auf downloads.sasd.de erscheinen soll, muss vor dem Artefakt-Eintrag auch ein Produktdatensatz in data/catalog/source/products.json angelegt werden. Der productSlug des Artefakts muss dann exakt mit dem slug des Produkts übereinstimmen.

```
{
 "id": "forgeevents",
 "slug": "forgeevents",
 "title": {
  "de": "ForgeEvents",
  "en": "ForgeEvents"
 },
 "summary": {
  "de": "Event-Plattform im SASD-Umfeld.",
  "en": "Event platform in the SASD ecosystem."
 },
 "description": {
  "de": "ForgeEvents dient als Grundlage für Event-Management, Planung und Veröffentlichung.",
  "en": "ForgeEvents provides a foundation for event management, planning and publication."
 },
 "productPageUrl": "https://software.sasd.de/products/forgeevents",
 "featured": true,
 "tags": ["events", "platform", "sasd"]
}
```

9. Validierung durchführen

Nach jeder Änderung an den Quell-JSON-Dateien sollte sofort die Validierung ausgeführt werden. Das Skript prüft insbesondere, ob Pflichtfelder vorhanden sind und ob die in path referenzierte Datei tatsächlich im Dateisystem existiert.

```
php cli/validate-catalog.php
```

Erwartetes Ergebnis

Im Erfolgsfall meldet das Skript: Catalog validation passed.

Im Fehlerfall wird mindestens ein Problem mit Bezug auf Feld oder Dateipfad gemeldet. Beheben Sie alle Fehler vollständig, bevor Sie den Katalog neu aufbauen.

10. Generierten Katalog neu aufbauen

Erst nach erfolgreicher Validierung wird der generierte Katalog aktualisiert. Beim Rebuild werden zusätzliche technische Informationen wie Dateiname, Dateigröße und SHA-256-Prüfsumme aufgenommen. Der generierte Katalog wird unter data/catalog/generated/catalog.json neu geschrieben.

```
php cli/rebuild-catalog.php

Generated catalog successfully.
Products: 3
Artifacts: 11
```

11. Lokale Funktionsprüfung

Nach dem Rebuild sollten Sie die Anwendung lokal starten und gezielt die betroffenen Seiten prüfen. Entscheidend ist nicht nur, dass kein PHP-Fehler auftritt, sondern dass das neue Artefakt in Suche, Listen, Produktseite und Detailansicht plausibel erscheint.

```
php -S 127.0.0.1:8080 -t public public/index.php
```

- Startseite prüfen: erscheint das Artefakt eventuell in „Neu & Aktualisiert“ oder „Beliebte Downloads“?
- Suche mit Produktname, Typ und Tags prüfen.
- Produktdetailseite öffnen und Zuordnung kontrollieren.
- Artefakt-Detailseite öffnen und Beschreibung, Version, Sprache und Datum gegenprüfen.
- Direkten Download anklicken und Dateiauslieferung testen.

12. Deployment in die Zielumgebung

Für die Veröffentlichung in der Zielumgebung müssen sowohl die neue Datei unter storage/files als auch die geänderten JSON-Dateien und gegebenenfalls der neu generierte catalog.json auf den

Zielserver gelangen. Halten Sie das Deployment konsistent: Code, Katalogquellen und Artefakte sollten zueinander passen.

- Repository-Änderungen committen und auf den Zielserver bringen.
- Neue Artefaktdateien synchronisieren.
- Auf dem Zielserver optional erneut validate-catalog.php und rebuild-catalog.php ausführen.
- Produktivseite im Browser testen, einschließlich Download-URL.

13. Mehrsprachigkeit sauber pflegen

Die Anwendung ist bereits für Deutsch und Englisch ausgelegt. Deshalb sollten Titel und Beschreibung eines Artefakts grundsätzlich immer für beide Sprachen gepflegt werden, selbst wenn der eigentliche Dateiinhalt nur in einer Sprache vorliegt. So bleibt die Oberfläche konsistent und Suchergebnisse wirken vollständig.

Aspekt	Empfehlung
Dateiinhalt	Kann einsprachig oder mehrsprachig sein.
language-Feld	Setzen Sie de oder en bei sprachspezifischen Dateien, multi bei sprachneutralen Paketen.
title / description	Immer mindestens mit de und en pflegen.
Weitere Sprachen später	Die Architektur erlaubt spätere Erweiterung um fr, es, pt, it, pl, tr, ar, hi, ko, zh und ja.

Wichtige Unterscheidung

Ein englisches PDF darf selbstverständlich language = "en" haben. Trotzdem sollte der Titel in den JSON-Metadaten sowohl auf Deutsch als auch auf Englisch formuliert werden, damit deutsche und englische Oberflächen beide sinnvoll funktionieren.

14. Sonderfall: Artefakt zu Trainingsmaterial oder Buch hinzufügen

Nicht jedes Artefakt gehört zu einem Produkt. Trainingsunterlagen und Buchbegleitmaterial können ohne productSlug arbeiten. In diesem Fall wird productSlug auf null gesetzt und die fachliche Einordnung primär über group, type, title und path gesteuert.

```
{
 "id": "cpp-training-2026-06-workbook",
 "slug": "cpp-training-2026-06-workbook",
 "productSlug": null,
```

```
  "group": "training",
  "type": "workbook",
  "title": {
    "de": "C++ Training Juni 2026 – Workbook",
    "en": "C++ Training June 2026 – Workbook"
  },
  "description": {
    "de": "Arbeitsunterlage für das SASD C++ Training im Juni 2026.",
    "en": "Workbook for the SASD C++ training in June 2026."
  },
  "version": "2026-06",
  "language": "de",
  "platform": "neutral",
  "status": "current",
  "releaseDate": "2026-06-04",
  "path": "training/cpp-course/2026-06/cpp-training-2026-06-workbook-de.pdf",
  "featured": true,
  "popular": false,
  "tags": ["training", "cpp", "workbook"]
}
```

15. Artefakt ändern, ersetzen oder archivieren

In der Praxis kommt es häufig vor, dass ein bestehendes Artefakt aktualisiert, durch eine neue Version ersetzt oder bewusst archiviert werden muss. Die sauberste Strategie ist in der Regel, neue Versionen als neue Einträge anzulegen, statt alte Datensätze zu überschreiben. So bleiben Historie, Prüfsummen und Downloadpfade klar nachvollziehbar.

- Neue Version: neuen Ordner und neuen Artefaktdatensatz anlegen.
- Alten Stand nicht überschreiben, sondern status auf deprecated oder archived setzen.
- Wenn ein Artefakt nicht mehr sichtbar sein soll, besser kontrolliert archivieren als still löschen.
- Physische Dateien nur dann löschen, wenn die Entfernung bewusst beschlossen und dokumentiert wurde.

16. Häufige Fehlerbilder und ihre Ursachen

Symptom	Wahrscheinliche Ursache	Abhilfe
Validierung meldet fehlende Datei	path stimmt nicht mit dem Dateisystem überein.	Dateipfad und Dateinamen exakt vergleichen, dann erneut validieren.
JSON kann nicht gelesen werden	Komma, Klammer oder Anführungszeichen fehlerhaft.	JSON-Syntax korrigieren; am besten Eintrag an bestehendem Muster ausrichten.

Symptom	Wahrscheinliche Ursache	Abhilfe
Artefakt erscheint nicht in der Produktseite	productSlug fehlt oder passt nicht zum Produkt-slug.	productSlug gegen products.json prüfen.
Download liefert 404 oder Fehlseite	Datei nicht deployed oder Pfad in Produktion abweichend.	Produktivdateien und storage/files synchronisieren, dann erneut testen.
Titel erscheint nur teilweise sinnvoll	de/en Texte unvollständig gepflegt.	Mehrsprachige Felder vollständig ausformulieren.

17. Checkliste vor Freigabe

- Datei liegt im richtigen Unterordner unter storage/files.
- Dateiname ist konsistent, stabil und ohne Leerzeichen.
- Neuer JSON-Eintrag in artifacts.json ist syntaktisch korrekt.
- title und description sind mindestens für de und en gepflegt.
- productSlug stimmt zum Produkt oder ist bewusst null.
- validate-catalog.php lief ohne Fehler durch.
- rebuild-catalog.php wurde erfolgreich ausgeführt.
- Lokaler Test von Suche, Detailseite und Download ist erfolgt.
- Deployment und Produktionsgegenprüfung wurden durchgeführt.

18. Empfohlener Minimalprozess für den Alltag

Wenn Sie künftig regelmäßig neue Artefakte veröffentlichen, empfiehlt sich ein immer gleicher Kurzprozess. Er reduziert Flüchtigkeitsfehler und macht die Pflege auch Monate später noch nachvollziehbar.

```
1. Datei benennen und ablegen
2. artifacts.json ergänzen
3. optional products.json ergänzen
4. php cli/validate-catalog.php
5. php cli/rebuild-catalog.php
6. lokal testen
7. deployen
8. produktiv prüfen
```

19. Fazit

Das Hinzufügen eines neuen Artefakts in downloads-sasd ist bewusst einfach gehalten, verlangt aber diszipliniertes Arbeiten an den richtigen Stellen. Wenn Dateiablage, JSON-Metadaten, Validierung, Rebuild und Test als zusammengehöriger Prozess verstanden werden, bleibt die Plattform trotz file-basierter Architektur robust, nachvollziehbar und später gut auf eine Datenbank migrierbar.